

ESE 561 · THEORY OF ARTIFICIAL INTELLIGENCE

From Uncertainty to Adversaries

Adversarial Search: Minimax and Alpha-Beta Pruning

M

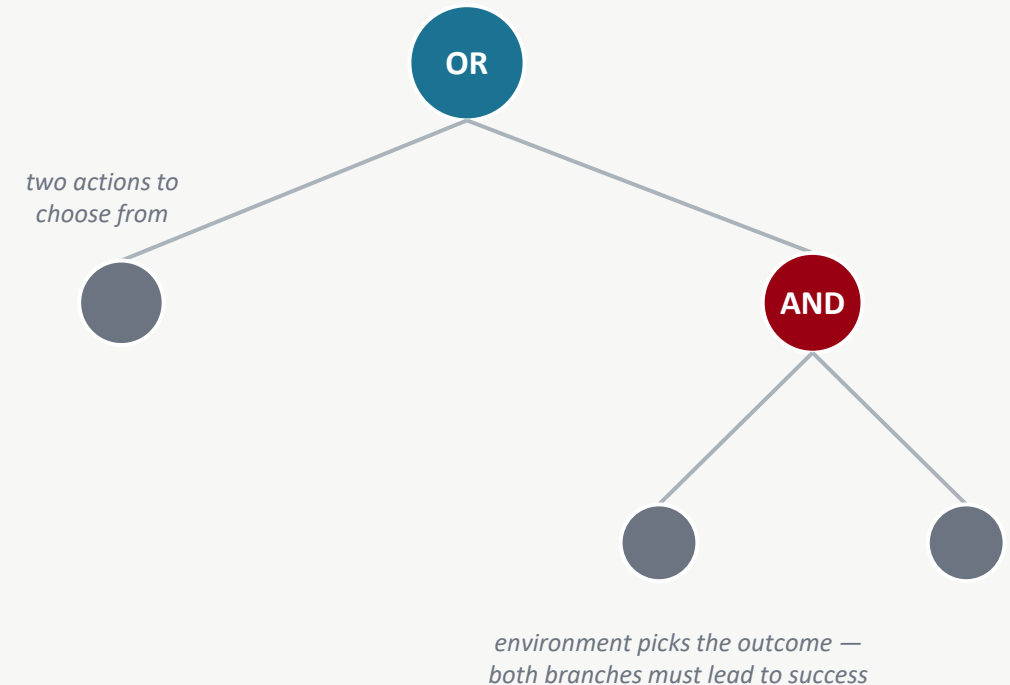
MAX — the agent whose utility we evaluate

m

MIN — the opponent, minimizing that same utility

When Actions Don't Have One Outcome

- Nondeterministic actions: $\text{Results}(s, a)$ returns a set of possible states, not one
- A fixed action sequence can fail — solutions must branch on what happens
- **AND–OR search trees handle the branching:**
 - OR-nodes — the agent chooses; one working branch is enough
 - AND-nodes — the environment decides; every branch must work



A solution is a tree of conditional decisions — “if this happens, do that.”

When You Can't See the Full State

- A belief state is the set of physical states consistent with what the agent knows
- Search shifts from states to belief states
- No sensors → one action sequence must work for every state in the belief set
- With sensing → belief shrinks after each percept, and the plan can branch on it
- **Solutions go from fixed sequences to conditional, percept-driven plans**

No sensors

One sequence guaranteed to work
for every possible starting state

With sensing

Predict → observe → update belief,
adapt the plan to each new percept

Is Playing an Opponent Just More Uncertainty?

- Nondeterminism assumes nature picks an outcome — indifferent, not out to get you
- In tic-tac-toe, does AND–OR search's “every outcome must work” framing fit an opponent?
- If you want to win, what would a contingent plan even look like?
- If winning isn't guaranteed, can you at least guarantee a draw?

X		O
	X	
O		X

Key question: is the opponent just another source of uncertainty — or something fundamentally different?

An Opponent Is Not Neutral Nature

- Nondeterminism: the environment's outcome is fixed by hidden dynamics, not by intent
- Adversarial search: another agent actively chooses outcomes to work against you
- That agent is assumed rational — it picks whichever response benefits itself
- **So evaluating one move means reasoning through the opponent's best counter-move, and then your best response to that, and so on**

A game is a search problem where part of the environment is another decision-maker.

Example: placing a mark may look strong now — but if it lets the opponent complete three in a row next turn, it's a bad move.

What Makes Something a “Game” in AI

1

Turn-taking agents

Multiple agents act, typically alternating

2

Conflicting objectives

What helps one player can hurt another

3

Joint outcomes

The result depends on both sides' decisions

4

Strategy, not sequence

A solution specifies a response to every situation, not one fixed plan

The environment is no longer passive — another agent actively shapes the future.

Games Differ Along Several Dimensions

- Deterministic or stochastic outcomes?
- Fully or partially observable information?
- Two players, or more than two?
- Zero-sum, or general objectives?

	Deterministic	Stochastic / Chance
Perfect information	Chess, Checkers, Go	Backgammon, Monopoly
Imperfect information	Battleship, Stratego	Bridge, Poker, Scrabble

Today: two-player, zero-sum, deterministic, perfect-information games — the top-left cell.

Zero-Sum vs. General Games

Zero-Sum Games

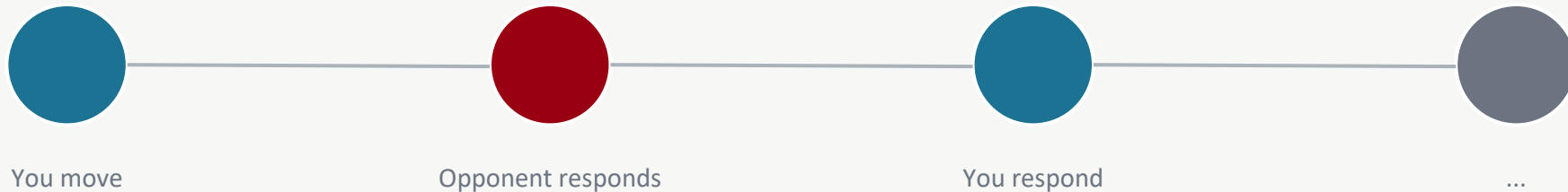
- Directly opposed objectives
- One player's gain equals the other's loss
- One maximizes utility, the other minimizes the same value

General Games

- Objectives may be independent
- Interests can partly conflict or partly align
- Cooperation, competition, coalitions can all appear

Backward Reasoning in Games

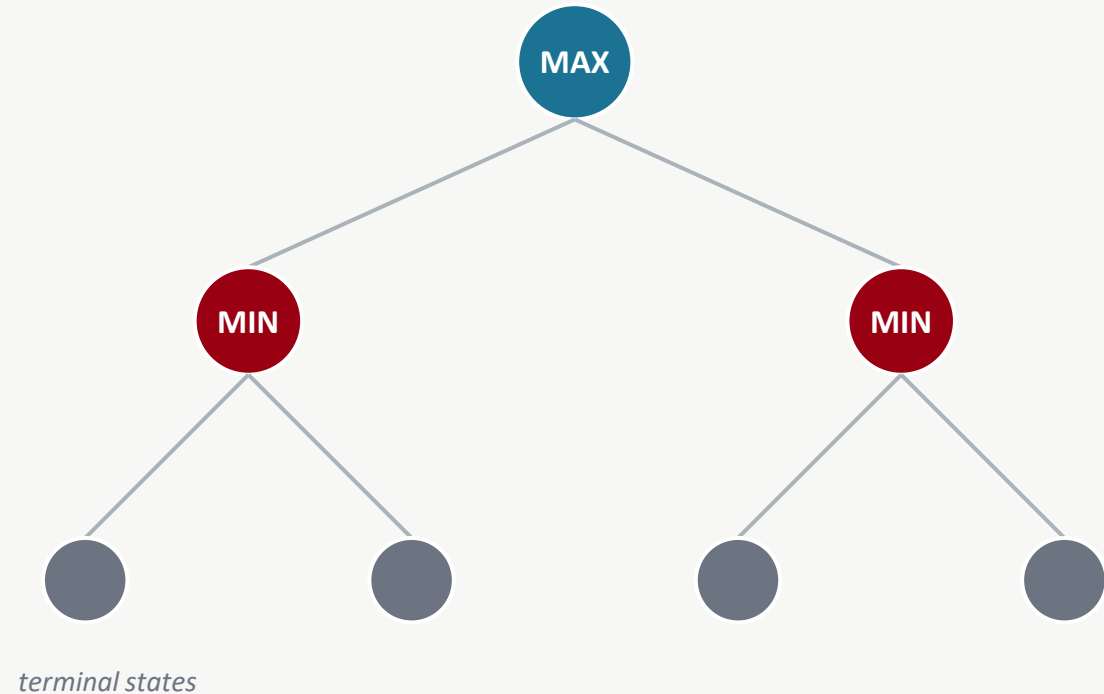
- Choosing a move isn't just about its immediate result
- You must consider how the opponent is likely to respond
- The opponent is assumed to respond in whatever way benefits itself
- **So evaluating one move means reasoning through future counter-moves — a nested process, worked out backward from the end of the game**



evaluated from the end of this chain, backward

Game Trees: MAX and MIN Levels

- Each node is a game state; each edge is a legal move
- Levels alternate between MAX and MIN
- MAX = the player whose utility we evaluate — picks the highest value
- MIN = the opponent — assumed to pick the lowest value
- **The tree expands until terminal (finished-game) states**



Minimax: Formal Definition

- Terminal states have known utility values
- At MAX nodes, take the child with the highest value
- At MIN nodes, take the child with the lowest value
- Internal node values propagate upward, recursively
- **Minimax computes the best guaranteed outcome under optimal adversarial play**

Tic-Tac-Toe Convention

+1 X wins

0 Draw

-1 O wins

Utility is always defined from MAX's (X's) perspective.

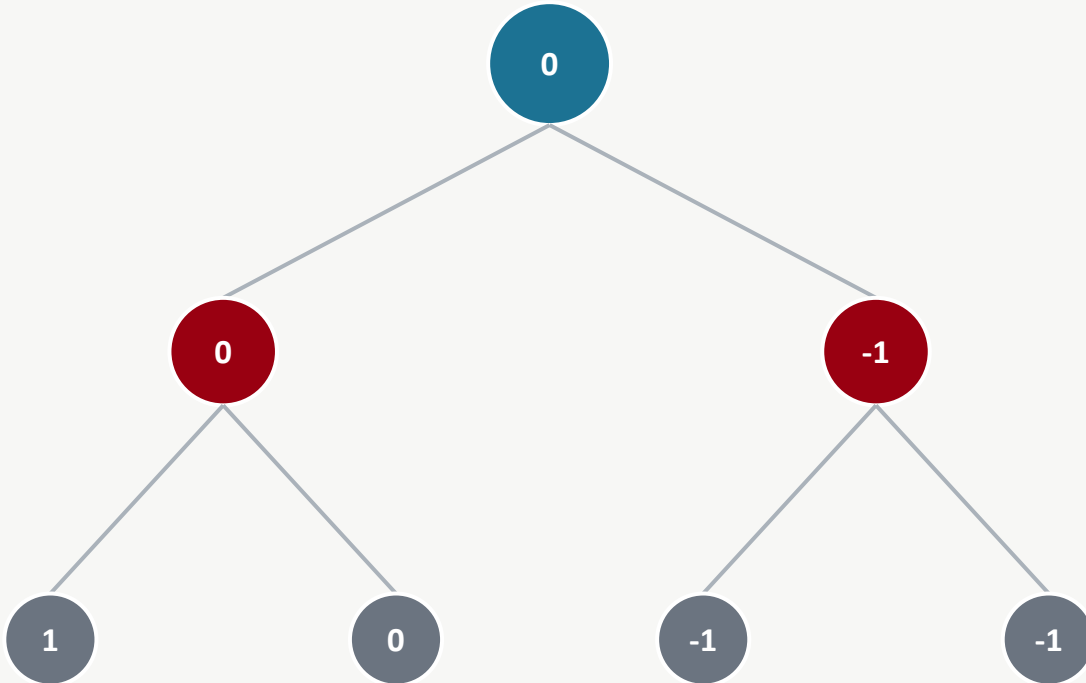
The Minimax Algorithm (Recursive)

```
MAX-VALUE(s):  
    if TERMINAL(s): return UTILITY(s)  
    v =  $-\infty$   
    for each a in ACTIONS(s):  
        v = max(v, MIN-VALUE(RESULT(s,a)))  
    return v
```

```
MIN-VALUE(s):  
    if TERMINAL(s): return UTILITY(s)  
    v =  $+\infty$   
    for each a in ACTIONS(s):  
        v = min(v, MAX-VALUE(RESULT(s,a)))  
    return v
```

- Depth-first traversal of the game tree
- Two mutually recursive functions, alternating by turn
- Terminal states return their utility directly
- Every other node aggregates its children's values

Worked Example: Scoring an Opening Move



MAX picks the max of its children ($0 \geq -1$) • MIN picks the min of its children

- Values propagate bottom-up: leaves $\rightarrow$ MIN nodes $\rightarrow$ root
- `estimate_move_value(move)` just reads off the root's value after placing that opening move
- **In real tic-tac-toe, every opening move minimaxes to exactly 0**
 - Perfect play on both sides always draws — no opening forces a win

Why Full Minimax Doesn't Scale

~35

average legal moves
per position in chess

~80

typical game length
(plies) in chess

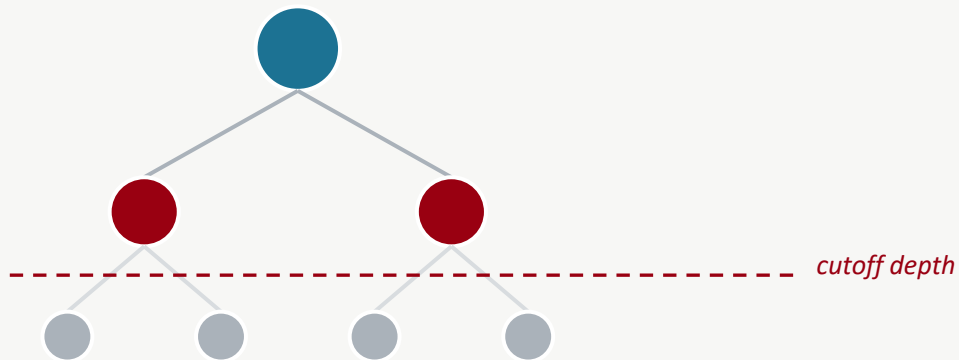
35^{80}

size of the full
game tree to search

Tic-tac-toe is small enough to solve exactly. Real games are not — we need to cut the search short and prune what we can.

Cutting Off Search: Evaluation Functions

- Search to a fixed depth limit, then stop before reaching terminal states
- Estimate the value of that cutoff state with an evaluation function instead of `Utility()`
- Risk: a shallow, imperfect estimate can make different moves look equally good — or flip the preferred move as depth changes
- **Iterative deepening runs this repeatedly at increasing depth limits, so there's always a best-found move ready if time runs out**



`eval(s)` estimates the value of a non-terminal cutoff state

The Intuition: Stop Looking Once It Can't Matter

- MAX keeps the best value found so far along this path (α)
- A branch worse than α will never be chosen by rational MAX — skip it
- MIN keeps the smallest value found so far along this path (β)
- A branch larger than β will never be chosen by rational MIN — skip it



MAX's best guaranteed value so far

raised whenever MAX finds something better



MIN's best guaranteed value so far

lowered whenever MIN finds something better

Prune whenever $\alpha \geq \beta$ — the result is provably unchanged

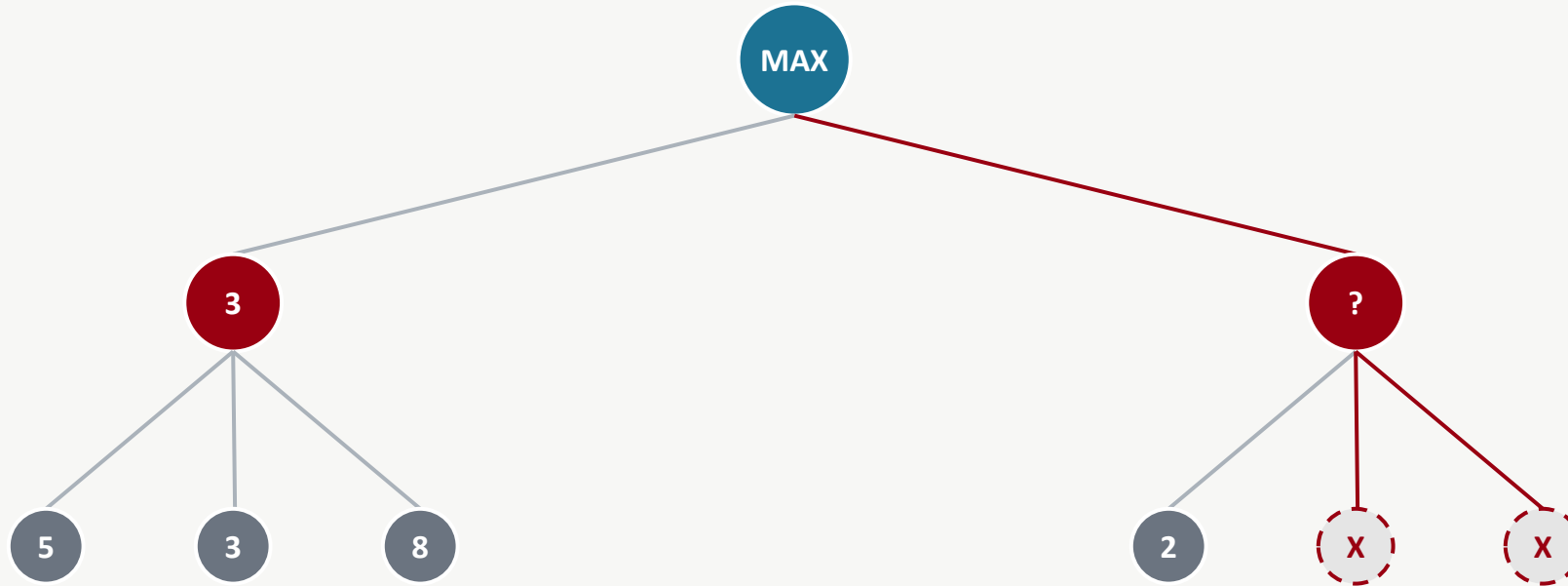
The Alpha-Beta Algorithm

```
MAX-VALUE(s,  $\alpha$ ,  $\beta$ ):  
  if TERMINAL(s): return UTILITY(s)  
  v =  $-\infty$   
  for each a in ACTIONS(s):  
    v = max(v, MIN-VALUE(RESULT(s,a),  $\alpha$ ,  $\beta$ ))  
    if v  $\geq$   $\beta$ : return v      # prune  
     $\alpha$  = max( $\alpha$ , v)  
  return v
```

```
MIN-VALUE(s,  $\alpha$ ,  $\beta$ ):  
  if TERMINAL(s): return UTILITY(s)  
  v =  $+\infty$   
  for each a in ACTIONS(s):  
    v = min(v, MAX-VALUE(RESULT(s,a),  $\alpha$ ,  $\beta$ ))  
    if v  $\leq$   $\alpha$ : return v      # prune  
     $\beta$  = min( $\beta$ , v)  
  return v
```

- Start with $\alpha = -\infty$, $\beta = +\infty$
- α , β pass down the current path
- MAX updates α only; MIN updates β only
- **Same result as plain minimax — just fewer nodes visited**

Worked Example: A Branch Gets Skipped



- Left subtree fully explored $\rightarrow$ MIN value = 3, so MAX's α becomes 3
- Right subtree's first child = 2
- Since $2 \leq \alpha(3)$, MIN can already guarantee ≤ 2 here — worse for MAX than the 3 already secured
- **The remaining children (marked x) are never evaluated**

Properties, Ordering, and What to Remember

Properties

- Minimax is complete and optimal against a rational opponent, but exponential in depth
- Alpha-beta returns exactly the same decision as minimax
- With a good move ordering, alpha-beta can search roughly twice as deep in the same time

Move Ordering

- Pruning only kicks in once useful bounds exist
- Examining strong moves first tightens α/β sooner $\rightarrow$ earlier pruning
- Practical sources of ordering: shallow-search results, domain heuristics, cached prior evaluations

Notebook: Score Every Tic-Tac-Toe Opening

- The game engine (board, valid moves, winner check) is already written for you
- You implement `minimax(board, player)` and `estimate_move_value(move)`
- Then rank all 9 opening moves by their exact game-theoretic value
- **Discussion: does the ranking match “corner is best” intuition, or does perfect play tell a different story?**

```
def minimax(board, player):  
    ...  
def estimate_move_value(move):  
    ...
```

Where We Landed Today

- Adversarial search extends state-space search to settings where a rational opponent shapes the outcome
- Minimax assumes optimal play on both sides and propagates utility backward through the game tree
- Full-depth minimax is infeasible for real games — evaluation functions plus iterative deepening make cutoff search practical
- Alpha-beta pruning eliminates branches that can't affect the decision, without changing the result
- **Together, these make adversarial search practical in large game spaces**

Beyond Path Search: Constraint Satisfaction

- So far, every solution we've searched for has been a path or a strategy — a sequence, or a tree of moves
- Many problems aren't about paths at all — they're about finding an assignment of values that satisfies a set of constraints
- Map coloring, scheduling, Sudoku, puzzle boards like N-Queens — all the same shape underneath
- **Constraint Satisfaction Problems (CSPs) make that internal structure — variables and constraints — explicit instead of hiding it inside an opaque state**

What Is a CSP?

1

Variables

$X_1, X_2, \dots, X_n$ — the things we need to decide

2

Domains

$D_1, D_2, \dots, D_n$ — the possible values each variable can take

3

Constraints

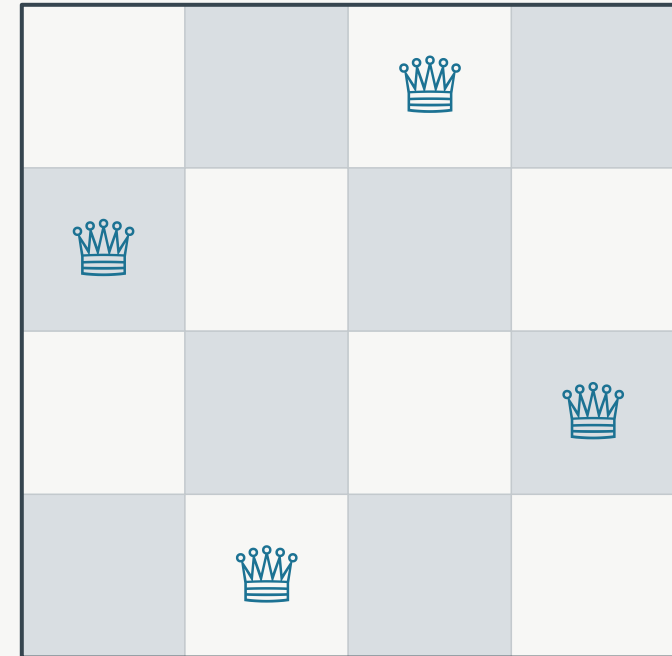
Restrictions on which combinations of values are allowed together

During search

- Partial assignment — only some variables assigned so far
- Consistent assignment — no constraint violated so far
- **A solution must be both complete and consistent**

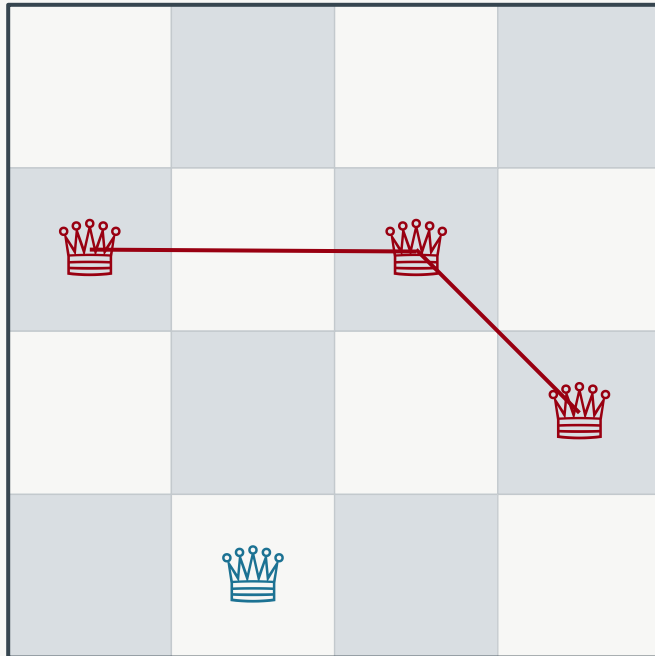
Example: N-Queens as a CSP

- Place N queens on an $N \times N$ board so that no two attack each other
- No two queens may share a row, a column, or a diagonal
- Variables: one per column, $Q_1 \dots Q_n$
- Domain of each Q_i : which row it occupies, $1 \dots N$
- Constraints: $Q_i \neq Q_j$, and $|Q_i - Q_j| \neq |i - j|$, for every pair $i \neq j$



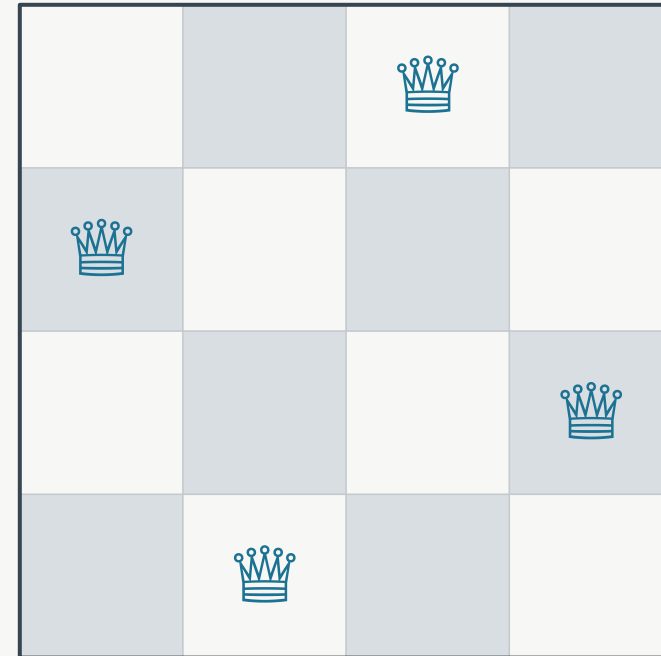
One valid solution

What a Constraint Violation Looks Like



Conflicting placement

Q_1 & Q_3 share a row • Q_3 & Q_4 share a diagonal

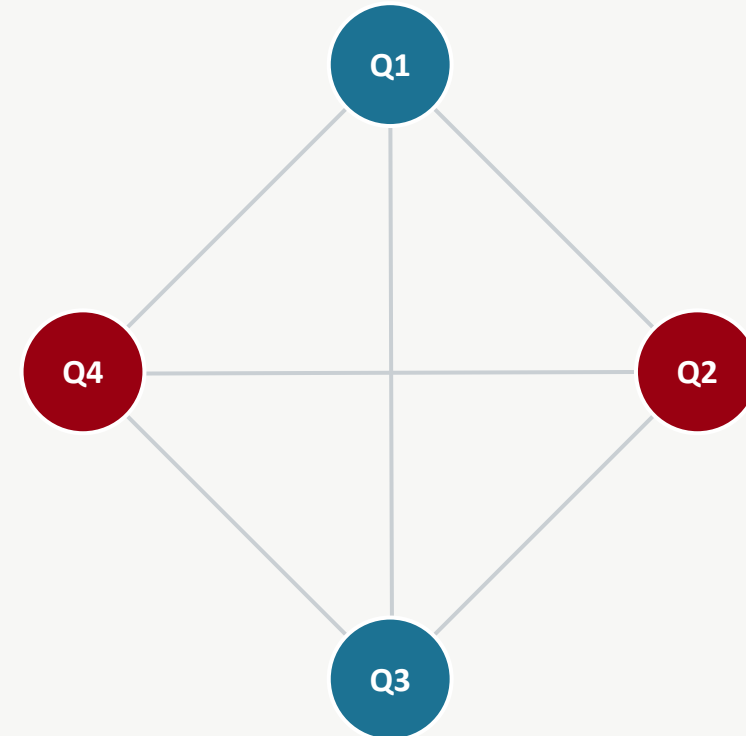


Valid solution

Complete and consistent — every constraint satisfied

Constraint Graphs

- For binary CSPs, we can draw a constraint graph
- Nodes represent variables; edges represent constraints between pairs of them
- CSP algorithms exploit this graph structure to reduce the search space
- **In N-Queens, every pair of queens constrains every other pair — so the constraint graph is fully connected**

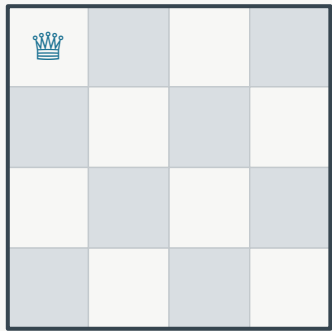


Backtracking Search

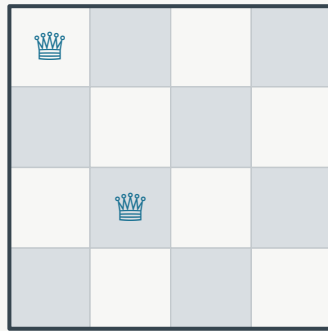
```
BACKTRACK(assignment, csp):  
    if assignment is complete:  
        return assignment  
    var = select unassigned variable  
    for each value in domain(var):  
        if value consistent with assignment:  
            add {var = value} to assignment  
            result = BACKTRACK(assignment, csp)  
            if result ≠ failure: return result  
            remove {var = value}    # backtrack  
    return failure
```

- Structured depth-first search over partial assignments
- Assign one variable at a time
- Check consistency before going deeper — don't wait until the end
- **When every value fails, undo the last assignment and try another — that's the “backtrack”**

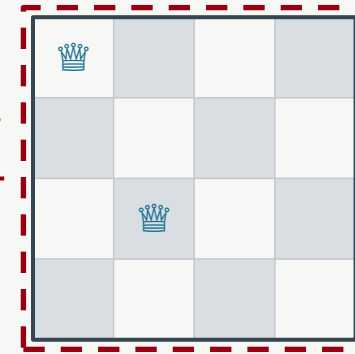
Worked Example: Backtracking on 4-Queens



Assign Q_1 = row 1



Q_2 : rows 1, 2 conflict
row 3 works



**Q_3 : every row conflicts
→ backtrack to Q_2**

← undo Q_3 , try its next value

- This is exactly what BACKTRACK does mechanically — no magic, just try, check, and undo on failure
- For 4-Queens specifically, this dead-end resolves after a couple more backtracks — a solution does exist (as we saw two slides ago)

Making Backtracking Smarter

Minimum Remaining Values (MRV)

- Which variable to assign next?
- Pick the one with the fewest legal values left
- Fails fast — finds dead-ends earlier instead of later

Least Constraining Value (LCV)

- Which value to try first for that variable?
- Pick the one that rules out the fewest options for everyone else
- Keeps future choices open as long as possible

Combined, these heuristics let backtracking solve N-Queens instances with $N \approx 1000$ — far beyond what naive backtracking could touch.